

fairGhosts – Ant Colony Controlled Ghosts for Ms. Pac-Man

Iris Hunkeler

Institute for Information Systems, School of Business
University of Applied Sciences and Arts
Northwestern Switzerland
Olten, Switzerland
iris.hunkeler@students.fhnw.ch

Rolf Dornberger

Institute for Information Systems, School of Business
University of Applied Sciences and Arts
Northwestern Switzerland
Basel, Switzerland
rolf.dornberger@fhnw.ch

Fabian Schär

Institute for Information Systems, School of Business
University of Applied Sciences and Arts
Northwestern Switzerland
Olten, Switzerland
fabian.schaer@students.fhnw.ch

Thomas Hanne

Institute for Information Systems, School of Business
University of Applied Sciences and Arts
Northwestern Switzerland
Olten, Switzerland
thomas.hanne@fhnw.ch

Abstract—A previously proposed ACO game agent controller for steering the ghosts in the computer game Ms. Pac-Man has been improved by reengineering and extending the original controller with an adapted ACO algorithm. The number of free parameters is reduced by 30% and their values are optimized using a genetic algorithm. The proposed new ACO controller, named “fairGhosts”, is tested against other available game agent controllers within the existing Ms. Pac-Man simulator framework. This controller reaches a performance comparable to the upper third of all proposed controllers. In certain cases, it beats even the best controllers. Our “fairGhost” algorithm, its implementation and functionality are discussed.

Keywords—Ant colony optimization, Ms. Pac-Man, game agent, game controller, swarm intelligence

I. INTRODUCTION

Heuristics and metaheuristics are used to find good solutions for complex optimization problems where exact methods fail to provide the optimal solution in a reasonable amount of time. Ant colony optimization (ACO) is such a metaheuristic which is inspired by the behavior of ants foraging for food sources. ACO uses an artificial ant colony that communicates indirectly through the use of artificial pheromone trails to create solutions. This metaheuristic can be applied to almost all optimization problems that can use an incremental approach to build a solution. The detailed theory behind ACO can be found in [1].

Video games have previously been used as proving-ground for computational intelligence applications. Amongst them is Ms. Pac-Man, a very popular pursuit-evasion game which follows simple rules, but requires complex strategies to win. This has made it especially popular among computational intelligence researchers. This popularity was only increased by

two different Ms. Pac-Man competitions: A screen capture competition held from 2007 to 2011 [2] and the Ms. Pac-Man vs. Ghosts competition held in 2011 and 2012, where the players directly interfaced with the game engine [3]. Various studies evaluated different strategies to develop game-agents for the game Ms. Pac-Man: In 2012, Pepels and Winands very successfully applied Real-Time Monte Carlo Tree Search to Ms. Pac-Man, and scored first and second places in two competitions that year [4]. Liberatore et al. analyzed a Genetic Algorithm with Lexicographic Ranking and obtained satisfactory results for a homogeneous team learning case [5]. Tan et al. built a controller based on the hill-climbing concept and simple feedforward neural networks [6]. An approach based on reinforcement learning was applied by Bom et al., resulting quickly in good playing behavior which was also transferable to other, not previously trained mazes [7]. Foderaro et al. implemented a model-based approach to compute optimal decision strategies which outperformed Beginner and Intermediate human players [8].

Recio et al. [9] propose an ACO based framework for developing Ms. Pac-Man game agents. They implemented game agent controllers both for Ms. Pac-Man as well as for the group of ghosts hunting her, and tested them at the Ms. Pac-Man vs. Ghosts competition [10] held during the IEEE World Congress on Computational Intelligence (WCCI 2012). Their research showed that swarm intelligence in general and ACO in particular can successfully be used in controlling game agents.

This paper aims at improving the ACO based game agent controller previously proposed for the ghosts in the game Ms. Pac-Man [9]. The goal is to adjust the proposed framework and to test the resulting game against other controllers developed for the Competition at the WCCI 2012 [11]. This is done to

show how effectively the ACO algorithm can be applied to this specific problem.

This paper is organized as follows. Section II describes the basic rules of the game Ms. Pac-Man as well as the additional restrictions defined by the competition Ms. Pac-Man vs. Ghosts. In Section III, the details of the implementation of “fairGhosts” – the ghost controller based on the framework proposed by Recio et al. [9] – are explained. Section IV outlines our main adaptations done to the framework; some of which were necessary due to assumptions we had to make, while others were done for optimization reasons. Section V discusses the parameter values used in “fairGhosts” and how they were obtained using a genetic algorithm. In Section VI our newly implemented controller is verified and tested against a selection of other controllers written for the simulator from WCCI 2012 [11]. The main findings and conclusion of this paper are then discussed in Section VII.

II. PROBLEM DESCRIPTION

A. Ms. Pac-Man Game

The game Ms. Pac-Man was released in 1981 with an almost identical gameplay as that of its predecessor Pac-Man [12]: Ms. Pac-Man is controlled by the player who guides her through a maze and earns points by eating pills which each are worth 10 points. When all pills are eaten, the level is finished and Ms. Pac-Man is taken to the next level. There are four differently colored ghosts (red = Blinky, pink = Pinky, blue = Inky and orange = Sue) that chase Ms. Pac-Man and try to catch her. If she is caught by one of them, she loses a life. In addition to the regular pills, there are four power pills on each level worth 50 points. If Ms. Pac-Man eats a power pill, the chasing ghosts slow down and lose their danger for a certain time during which they can be eaten now by Ms. Pac-Man for extra points; the first ghost is worth 200 points and each consecutively eaten ghost doubles those points. In addition, the

power pill also triggers a global reversal event which forces the ghosts to continue their path in the opposite direction. Each maze contains two fruit bonuses that yield extra points when eaten. Ms. Pac-Man starts with three lives and earns an extra live for every 10'000 points she achieves. If she has lost all lives, the game is over. One of the main changes from Pac-Man to Ms. Pac-Man was a redesign of the ghosts' behavior:

In Ms. Pac-Man, they move in a more complex and non-deterministic way. This makes it impossible to learn an optimal path to beat the game.

B. Ms. Pac-Man vs. Ghost Competition

The Ms. Pac-Man vs. Ghost Competition [10] offers a simulator version of the game where controllers for Ms. Pac-Man as well as for the ghosts can be played against each other. The simulator implements all features from the Ms. Pac-Man game except the fruit bonus points; they will thus not be considered in this paper. In the competition, the simulator restricts the response time for a next move of both Ms. Pac-Man and the ghosts to 40ms. If a controller does not respond in time, the game advances using the last chosen action or - if this is not a legal move - randomly chooses an action. The goal of the Ms. Pac-Man agent is to achieve as many points as possible whereas the target of the ghosts is to minimize her points.

Since they outnumber Ms. Pac-Man some restrictions are put on the movements of the ghosts: They can never turn back on themselves willingly and thus can only choose their path when they reach a junction. The occurrence of a global reversal event (which has a probability of 0.0015) forces the ghosts to go in the opposite direction of their current path.

In the 2011 version of the simulator, which is the one used by [9], a level ends after 3000 time steps and half of the remaining points are awarded to Ms. Pac-Man. This was intended to keep the ghosts from hovering around the power pills instead of chasing Ms. Pac-Man. However, for the 2012 competition the maximum of time steps for a level was raised

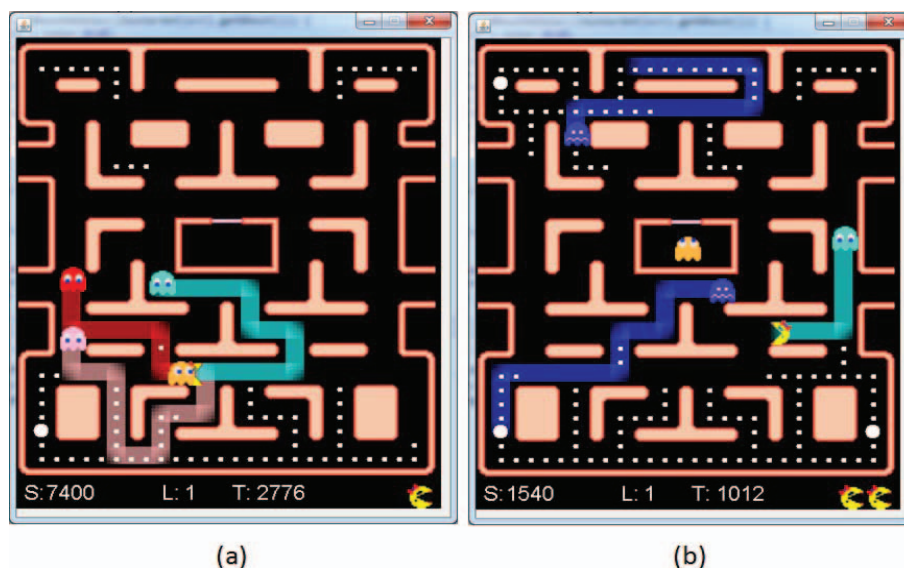


Fig 1: Screenshots showing different modes of the fairGhosts controller steering the ghosts: (a) four ghosts chasing Ms. Pac-Man choosing different paths where possible and (b) two edible ghosts trying to get away from catching Ms. Pac-Man while one ghost is on her chase.

to 4000 and no points were awarded if the level is advanced automatically. For our research, we chose to use the most currently available simulator version from 2012.

III. ACO GHOST CONTROLLER “FAIRGHOSTS”

This section describes the algorithm of “fairGhosts”, which is largely based on the framework proposed by [9]. The adaptations to the original framework are listed in Section IV. Fig. 1 shows different modes of “fairGhosts” in action: (a) The “fairGhosts” controller is steering four ghosts chasing Ms. Pac-Man by choosing different paths where possible. (b) Two edible ghosts are trying to get away from catching Ms. Pac-Man while one ghost is on her chase. The lines related to each ghost are the new computed paths, the ghosts intend to go.

A. Setup of the Graph for ACO

To apply ACO to Ms. Pac-Man, the game is considered a graph $G = (N, E)$ with N being all possible nodes and E the edges that connect the nodes. The game agents as well as the ants move along this graph. While an ant k of type x (k^x) is moving around, it collects information about its visited nodes S_k^x until its individual stopping criterion is reached. Ants communicate through the use of pheromones that they deposit on the edges they pass. Ants consider the pheromones on the edges adjoining their current node when choosing a next move and deposit pheromones as a guide for other ants.

B. Ant Setup

“fairGhosts” uses two kinds of ants to reach its goal of minimizing the score of Ms. Pac-Man: *ExplorerAnts* to find the possible next moves of Ms. Pac-Man and *HunterAnts* to guide the ghosts to catch Ms. Pac-Man or to avoid her if a ghost is in edible mode. Before the ants can be sent out, the graph is initialized with the minimum pheromone level τ_0 for all pheromone types. *ExplorerAnts* k^e are the first ants to be launched.

C. ExplorerAnt Logic

An iteration of *ExplorerAnts* includes an ant starting from each possible next node of Ms. Pac-Man. An *ExplorerAnt* can move to any adjacent node except the ones it has already visited. This also means that it cannot turn back on itself. Following the idea of mutation used in methods belonging to evolutionary computation, *ExplorerAnt* can randomly choose the next node instead of using the predefined one. Thus, a random number from $[0, 1]$ is chosen: If the number is larger than the *ExplorerAnts* exploration rate q_0^e the ant follows the path with the highest pheromone level; otherwise it randomly chooses a next node. The *ExplorerAnt* reaches its stopping criterion if it has either travelled the maximum allowed distance $maxDist^e$, has reached a point where a ghost was earlier or if there are no more unvisited nodes in its neighborhood.

After all *ExplorerAnts* of an iteration have reached their stopping criterion, the solution quality Q_k^e of each ant is evaluated using

$$Q_k^e = \sum_{i,j} \frac{\delta(n_j, g)}{\delta(n_i, n_j)^2} \quad (1)$$

where $\delta(n_i, n_j)$ represents the distance between the initial and current node of ant k^e and $\delta(n_j, g)$ refers to the distance between the current node of ant k^e and the nearest ghost. Afterwards, the global pheromone update for all *ExplorerAnts* of this iteration is triggered: The pheromone amount τ_{ij}^e is modified on all edges of the graph G that have been visited by the ant according to S_k^e with

$$\tau_{ij}^e(t+1) = \tau_{ij}^e(t) + \frac{Q_k^e}{Q_{bi}^e} \quad (2)$$

where Q_{bi}^e is representing the solution quality of the best *ExplorerAnt* in this iteration. Then, the next iteration is launched on the graph with the modified pheromone levels until the maximum number of allowed *ExplorerAnts* $maxAnt^e$ has been reached. After all *ExplorerAnts* have finished their execution, *HunterAnts* k^h are launched.

D. HunterAnt Logic

The paths of the ghosts are estimated in order of their closeness to Ms. Pac-Man; meaning the closest ghost first decides on its direction, followed by the second closest ghost and so on. For a ghost to decide on its direction, *HunterAnts* are created on all next possible nodes of the ghost. When a *HunterAnt* needs to choose a next node, all neighboring nodes which have not yet been visited and are not opposite the ant's last move are possible candidates. To choose a node, the desirability D_{ij} of each candidate node is calculated with

$$D_{ij} = (\tau_{ij}^e(t) + \tau_{ij}^h(t)) * \eta_{ij}^h(t) \quad (3)$$

$$\eta_{ij}^h(t) = \begin{cases} \frac{1}{\delta(n_i, n_{pm})} & \text{if ghost is not edible} \\ \delta(n_i, n_{pm}) & \text{if ghost is edible} \end{cases} \quad (4)$$

where $\tau_{ij}^e(t)$ and $\tau_{ij}^h(t)$ stand for the pheromone level of *ExplorerAnts* respectively *HunterAnts* and $\eta_{ij}^h(t)$ represents the heuristic value of a certain node based on the ghost's current status. Additionally, n_i represents the current node of the *HunterAnt* and $\delta(n_i, n_{pm})$ the distance between Ms. Pac-Man and node n_i . The heuristic value stands for the general attractiveness of a node: When a ghost is in chasing mode, the nodes closer to Ms. Pac-Man have a higher attractiveness than the nodes further away from her and vice versa if the ghost is in edible mode.

A *HunterAnt* travels the graph until it meets one of its stopping criteria: It has either travelled the maximum allowed distance $maxDist^h$, it has reached Ms. Pac-Man or there are no next candidate nodes. At this point, the solution quality of each *HunterAnt* in the iteration will be evaluated using

$$Q_k^h = \sum_{i,j} \frac{\eta_{ij}^h(t)}{\delta(n_i, n_j)} * \tau_{ij}^e(t) \quad (5)$$

Only the best *HunterAnt* of this iteration then executes the global pheromone update where the pheromone amount τ_{ij}^h is modified on all edges that were visited by that ant using

$$\tau_{ij}^h(t+1) = (1 - \alpha^h) * \tau_{ij}^h(t) + \alpha^h * Q_k^h \quad (6)$$

where α^h stands for the updating rate of HunterAnts. Afterwards, the next iteration of HunterAnts is launched until the maximum number of allowed HunterAnts $maxAnt^h$ has been reached.

After all iterations are finished, the HunterAnt with the highest solution quality Q_k^h of all iterations is selected as Q_b^h . The ghost will choose its next move to reach the node where Q_b^h has started. To make this path less attractive to other ghosts and give them more incentive to find an alternative way, the pheromone level of the explorer ants $\tau_{ij}^e(t)$ will be set to 0 on the path of Q_b^h . Due to this, the ghosts are more likely to be able to corner Ms. Pac-Man instead of just trailing her all on the same path. These ideas are summarized in Algorithm 1.

Algorithm 1: Pseudocode of “fairGhosts”. The algorithm returns a list with the next action for each GHOST.

```

totalNumberOfExplorerAnts = 0;
repeat
    initialize ExplorerAnts for iteration i
    totalNumberOfExplorerAnts += ExplorerAnts.size
    run ExplorerAnts until stopping criteria reached
    evaluate Solution Quality of ExplorerAnts
    do Pheromone Update for ExplorerAnts
until totalNumberOfExplorerAnts >= maxExplorerAnts
for all GHOST do
    totalNumberOfHunterAnts = 0;
    repeat
        initialize HunterAnts for iteration i
        totalNumberOfHunterAnts += HunterAnts.size
        run HunterAnts until stopping criteria reached
        evaluate Solution Quality of HunterAnts
        do Pheromone Update and Evaporation for best
        HunterAnt of iteration i
    until totalNumberOfHunterAnts >= maxHunterAnts
    find best HunterAnt for all iterations
    remove ExplorerAnt pheromones on path of best
    HunterAnt
    Actions ← chose move for GHOST based on best
    HunterAnt
end for
return Actions

```

E. Parameters

The “fairGhosts” algorithm is configurable by the set of parameters described in Table I. The values for these parameters will be optimized using a genetic algorithm which is described in Section V.

TABLE I. FAIRGHOSTS PARAMETER LIST

Parameter	Name
$maxAnt^e$	Maximum number of ExplorerAnts
$maxDist^e$	Maximum visited nodes of ExplorerAnts
q_0^e	Exploration rate for ExplorerAnts

$maxAnt^h$	Maximum number of HunterAnts
$maxDist^h$	Maximum visited nodes of HunterAnts
α^h	Updating rate for HunterAnts
τ_0	Minimum Pheromone level

IV. ADAPTATIONS TO THE FRAMEWORK

In several aspects, the ACO ghost controller “fairGhosts” described in Section III differs from the one proposed by Recio et al. [9]. Some assumptions were necessary, because during the implementation of the controller it became clear that some information is either missing, unclear, or possibly wrong. Some adaptations were implemented in order to integrate the “fairGhosts” algorithm, as shown in Algorithm 1, into the original ACO ghost controller. Additionally, some changes were made for performance reasons. This section describes the most important areas where we chose to make assumptions or adaptations.

A. Adaptations in the Setup

Minimum Pheromone Level τ_0 : Five different pheromone types are used: One for the ExplorerAnt and one for each of the ghosts. ExplorerAnts are only influenced by their own pheromones, whereas HunterAnts are influenced by their own pheromones and the ExplorerAnt pheromones. In the initialization of the graph before any ants are created, the pheromone level for each pheromone type is initialized to τ_0 . In [9] the minimum pheromone level is mentioned as a parameter but not explained in detail and no value is given for it.

Order of the ghosts in optimization: The optimization of the ghosts is based on their distance to Ms. Pac-Man. In [9] no specified order is visible.

B. Simplified ExplorerAnts

“fairGhosts” uses a strongly simplified version of the ExplorerAnts. Recio et al. [9] use the ExplorerAnts to try to anticipate the most likely moves of Ms. Pac-Man. This idea was completely abandoned in the controller “fairGhosts” proposed by this paper, because we did not want to spend resources on anticipating moves that might or might not happen; especially because Ms. Pac-Man is much more flexible than the ghosts are. Because of this, we did the following adaptations:

Pheromone evaporation: Contrary to [9], there is no pheromone evaporation for ExplorerAnt pheromones in “fairGhosts”, because we could not identify any advantages by doing so.

Pheromone deposition: All ExplorerAnts deposit pheromones, while in [9] only the best ExplorerAnt of each iteration is used for pheromone deposition. The reason is that the sum of all pheromone deposition forms the best path to follow.

Behavior near power pill: Recio et al. [9] implemented special considerations when Ms. Pac-Man is near a power pill.

These were not implemented for “fairGhosts” as a delineation for our algorithm.

C. Adapted HunterAnts

Choice of the next node: The logic of HunterAnts when choosing a next node has been adapted: In the framework from [9], the HunterAnts either move to the edge with the highest amount of pheromone or with a probabilistic rule based on the pheromone amount. In the controller proposed by our paper, the HunterAnts always move based on a probabilistic rule, where the probabilities are calculated using the desirability D_{ij} of a next node.

Heuristic value of the ghosts: To properly calculate the desirability D_{ij} , the calculation of the heuristic value was adapted as well. Recio et al. [9] define the heuristic value for a HunterAnt as

$$\eta_{ij}^h(t) = \begin{cases} \frac{1}{\delta(n_i, n_{pm})} & \text{if edible} \\ \delta(n_i, n_{pm}) & \text{otherwise} \end{cases} \quad (7)$$

where n_i represents the current node of the HunterAnt and $\delta(n_i, n_{pm})$ the distance between Ms. Pac-Man and node n_i . The heuristic value in ACO typically represents information about the problem instance which cannot be provided directly by ants. It often is the cost; in our case, however, it represents the attractiveness of a certain node: If a ghost is in its normal (i.e. not edible) state, nodes that are closer to Ms. Pac-Man are always more attractive than nodes further away. This means that the smaller $\delta(n_i, n_{pm})$, the more attractive is a certain node. To achieve this, we can use the inverse of $\delta(n_i, n_{pm})$. Only when a ghost is edible, the nodes further away from Ms. Pac-Man – i.e. with a bigger $\delta(n_i, n_{pm})$ – become more desirable. In the definition of the heuristic value from Recio et al. these two states appear to be switched around. For our controller implementation, we chose to use the corrected version as described in (4).

D. Reduced Number of Parameters

Due to the adapted logic, the number of parameters used in our implementation could be reduced: [9] used ten parameters for their “ghostAnt” implementation while “fairGhosts” only needs the seven parameters described in Table I.

V. OPTIMIZATION OF CONTROLLER PARAMETERS

The implementation of the “fairGhosts” controller provides the seven parameters as described in Table I to significantly influence the behavior of the ants and thus affect the quality of the next ghost-moves that are calculated. In order to get a set of optimized parameter values, an evolutionary algorithm (EA), i.e. a genetic algorithm, was applied. In a generic way, the algorithm starts with a set of candidate solutions (represented by chromosomes) which is called the population. For each chromosome in the population, a fitness value is calculated in order to determine which chromosomes will be selected for a potential crossover and mutation to create the next generation. The desired effect is that this new population in the next generation will be better than the previous one.

In “fairGhosts”, a solution is based on the definition of all seven parameters which each can be seen as individual genes of a seven-gene chromosome. A population therefore consists of several combinations of parameters for which the fitness value has to be calculated.

Since the behaviors of both the Ms. Pac-Man and the ghost controllers are non-deterministic, a single execution of the simulator can result in game scores with big variances. In order to mitigate this, each chromosome (i.e. a set of parameters) is used for 50 executions of the simulator to calculate its fitness value. Because the ghosts strive to minimize Ms. Pac-Man’s score, a good parameter set results in a small average game score. However, the genetic algorithm expects a high value for a good parameter set. To achieve a valid fitness value, the average score of the 50 executions was subtracted from 100’000.

The software framework JGAP (Java Genetic Algorithms Package) [13] was used to build the genetic algorithm that is put on top of the simulator [10]. JGAP is based on a generational evolution model. For the configuration, the default values for the crossover rate (0.35), the mutation rate (12, i.e. $1/12 = 0.0833$) and the natural selector (0.9, i.e. 90% of the top population) were used. All parameters (i.e. genes) are subject to certain constraints, both in their minimum and maximum value as well as in the step type as indicated by Table II. JGAP automatically uses the smallest possible step size for the defined step type.

TABLE II. FAIRGHOSTS PARAMETER OPTIMIZATION

Parameter	Limits	Step type
$maxAnt^e$	[5, 30]	Integer
$maxDist^e$	[10, 100]	Integer
q_0^e	[0, 1]	Double
$maxAnt^h$	[5, 40]	Integer
$maxDist^h$	[50, 200]	Integer
α^h	[0, 1]	Double
τ_0	[0, 2]	Double

A single Ms. Pac-Man controller had to be chosen to be used in the genetic algorithm. This controller should be high-ranking, but also high performing to ensure that the optimization does not require an extensive amount of time. In order to find a good balance between the total run time and the number of potential solutions, a population size of 50 chromosomes was defined to be used in 20 generations. Therefore, we selected the best ranked available controller which could run 50 executions against “fairGhosts” with eight parallel games in less than 10 minutes. This criterion resulted in the selection of the controller called “ghostbuster” for the parameter optimization.

TABLE III. SUMMARY OF COMPETITION ROUNDS. SCORES REPRESENT MEAN VALUES IN 50 EXECUTIONS

		Ghosts controllers								
		best	(3)	(2)	(1)	(8)	(9)	(11)	(19)	worst
			Memetix	GreenTea	eiisolver	rcpinto	fairGhosts	stefan	egreavette	Legacy2
best Ms. Pac-Man controllers	(4) Memetix		5'078	5'154	5'017	11'837	14'076	17'653	21'007	51'320
	(1) eiisolver		5'988	6'600	5'659	10'940	15'791	29'610	20'947	42'150
	(5) ghostbuster		5'317	6'657	5'335	9'283	9'746	16'436	18'487	48'074
	(9) egreavette		3'948	4'239	4'157	9'161	11'119	20'790	17'078	43'725
	(15) stefan		4'274	4'197	6'600	7'181	6'571	6'529	9'619	14'752
	(16) MinimaxRampage		3'415	4'296	4'211	8'641	6'362	5'559	13'647	21'664
	(10) rcpinto		2'842	2'634	2'912	5'211	4'460	2'370	7'286	10'395
	(22) Kazzim		3'012	2'109	2'976	4'238	4'593	3'910	4'947	4'290
	(34) StarterPacman		2'755	2'445	3'943	3'810	2'466	1'732	2'023	4'164
	Average		4'070	4'259	4'534	7'811	8'354	11'621	12'782	26'726
worst										46'333

The optimization was executed using a multithreaded setup, allowing eight games to be run simultaneously on a quad-core Intel® 3.5 GHz processor machine with 32 GB RAM, running on a 64-bit Windows 7®. The JDK was loaded into a RAM disk to further reduce execution time. The average required time for one generation was 293 minutes, resulting in an overall optimization time of a little more than 100 hours (4.2 days). As visible in Fig. 2, the biggest improvements were achieved in the first few generations, whereas afterwards no further improvements happened.

Fig 2 shows the average scores in 50 executions of the GA with its corresponding standard deviation of the sample size and the score of the best chromosome of each population. The best average score of 7'862 from a single chromosome against the Ms. Pac-Man controller “ghostbuster” was achieved in generation 10 with the following parameter values:

$$\begin{aligned}
 \max Ant^e &= 25 \\
 \max Dist^e &= 74 \\
 q_0^e &= 0.351436473318242 \\
 \max Ant^h &= 30 \\
 \max Dist^h &= 75 \\
 \alpha^h &= 0.018353682449733 \\
 \tau_0 &= 1.75868092119345
 \end{aligned}$$

VI. TEST CASES FOR VERIFICATION

Originally, we planned to verify “fairGhosts” by comparing it to the results of “ghostAnt” from [9]. However, “ghostAnt” was implemented for the simulator of the competition at CEC 2011 [14] while “fairGhosts” uses the most recent version of the simulator from WCCI 2012 [11] since no older version was available. Unfortunately, there has been a major change in the simulator between these two versions due to which the two ghost controllers cannot be compared with the same Ms. Pac-Man simulation framework. This effectively renders “fairGhosts” and “ghostAnt” incomparable.

In order to get an estimate of the quality of “fairGhosts”, the controller was challenged with a selection of Ms. Pac-Man

controllers and compared them with ghost controllers, both written for the simulator [10] of the WCCI 2012 competition [11]. In total, 63 Ms. Pac-Man and 55 ghost controllers were ranked in the WCCI 2012 competition [11]. Our selection of ghost and Ms. Pac-Man controllers includes a mix of high and middle ranked ones.

Although we had access to some of the controllers, most of them were either not available to us or only in a non-functioning version. Thus, the selection for comparison was strongly limited. Following descending rank, we selected the top 8 Ms. Pac-Man controllers available to us, out of which 5 are in the top 10, and complemented it with the “StarterPacMan” that is shipped with the simulator.

As for the ghost controllers, the list to choose from was very limited as well for the same reasons. Again following descending rank, the top 6 available ghost controllers were selected, out of which 5 are in the top 10, including “eiisolver” [15], “GreenTea” [16] and “Memetix” [17]. These controllers are complemented by two lower ranking controllers: “Legacy2TheReckoning” that is part of the simulator and

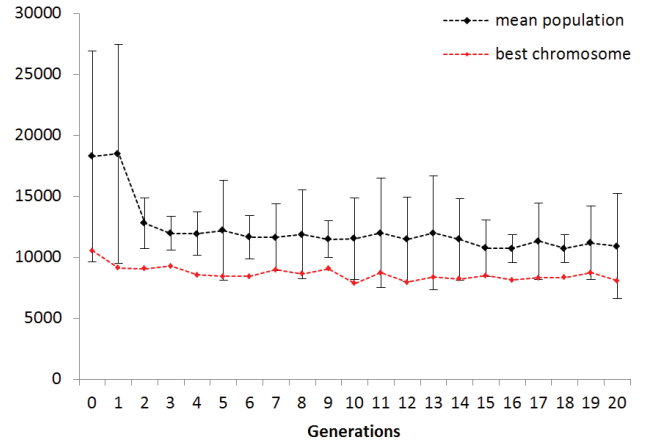


Fig 2: Average scores of the population (black) and best chromosome (red) in 50 executions of the GA for “fairGhosts”, the error bars represent standard deviations

“FC99”. We chose not only the best ghost controllers for comparison to facilitate a more complete rating of “fairGhosts”.

This selection of controllers gives an indication of the possible rank “fairGhosts” could have achieved at the competition.

A. Challenge and Comparison of “fairGhosts” with other Controllers from WCCI 2012 Competition

Since the controllers only have 40ms allocated processing time, they often need to sacrifice accuracy in order to stay within that time slot. This causes a considerable variance in the performance of the controllers as they do not necessarily reach an optimal solution in that time. To mitigate this fact somewhat and to make the controllers better comparable, each pair of Ms. Pac-Man and ghost controller competed with each other 50 times out of which the overall average score was taken for comparison. The result of this competition can be found in Table III, which shows the Ms. Pac-Man controllers on the vertical and the ghost controllers on the horizontal axis. They are ordered based on their average performance over all combinations, where the best Ms. Pac-Man controller can be found at the top and the best ghost controller in the most left column. The number in parenthesis is the rank a controller achieved in the competition WCCI 2012 [11].

Interestingly, the performance depends heavily on the specific combination of controllers: Such an example is the combination of the two ghost controllers “stefan” and “Memetix”: Against the “StarterPacman”, the lower ranked controller “stefan” achieved roughly a 50% better score than “Memetix”, whereas against the “eiisolver” Ms. Pac-Man, “stefan” was about five times worse than “Memetix”.

Based on these results, “fairGhosts” would have had good chances to end up among the top 10 of ghost controllers at the Ms. Pac-Man vs. Ghost Competition at the WCCI 2012.

VII. CONCLUSION

The framework proposed by Recio et al. [9] has been used to re-implement an adjusted game agent controller for the ghosts in the game Ms. Pac-Man. Even though not all parameters were quantified and not all functions explained in full detail, with some simplification and interpretation we have rebuilt the basic structure of the ACO framework and extended it so that our new “fairGhost” controller is now able to make meaningful ant-based decisions to achieve comparably good results in the Ms. Pac-Man simulator.

Unfortunately, both the original parameter set as well as the implementation of “ghostAnt” from Recio et al. [9] are not retrievable anymore. Thus, a direct comparison of our new “fairGhosts” with the original “ghostAnt” controller is not feasible and we cannot determine the exact influence of our adaptations and improvements as described in Section IV. However, the comparison with a selection of controllers from the WCCI 2012 competition showed that our “fairGhosts” is capable of achieving good results, positioned in the upper third of the proposed controllers. As shown in Table II, the

algorithm would have had realistic chances of ending up among the top 10 ghost controllers. It can be assumed therefore that ACO is capable of generating above-average decisions.

Based on the results obtained in this paper, ant colony optimization has clearly shown that it is a viable option for the development of a decision-making bot in a dynamically changing environment.

REFERENCES

- [1] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA: MIT Press, 2004.
- [2] “Ms Pac-Man Competition.” [Online, last accessed 7.4.2016]. Available: <http://dces.essex.ac.uk/staff/sml/pacman/PacManContest.html>
- [3] P. Rohlfshagen and S. M. Lucas, “Ms Pac-Man versus Ghost Team CEC 2011 competition,” in 2011 IEEE Congress on Evolutionary Computation (CEC), 2011, pp. 70–77.
- [4] T. Pepels, M. H. M. Winands, and M. Lanctot, “Real-Time Monte Carlo Tree Search in Ms Pac-Man,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 6, no. 3, pp. 245–257, Sep. 2014.
- [5] F. Liberatore, A. Mora, P. Castillo, and J. Merelo, “Comparing Heterogeneous and Homogeneous Flocking Strategies for the Ghost Team in the Game of Ms. Pac-Man,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. PP, no. 99, pp. 1–1, 2015.
- [6] T. G. Tan, J. Teo, and P. Anthony, “A simple heuristic search method for the automatic generation of neural-based game artificial intelligence architectures in Ms. Pac-Man,” in 2010 10th International Conference on Information Sciences Signal Processing and their Applications (ISSPA), 2010, pp. 753–756.
- [7] L. Bom, R. Henken, and M. Wiering, “Reinforcement learning to train Ms. Pac-Man using higher-order action-relative inputs,” in 2013 IEEE Symposium on Adaptive Dynamic Programming And Reinforcement Learning (ADPRL), 2013, pp. 156–163.
- [8] G. Foderaro, A. Swingler, and S. Ferrari, “A Model-based Approach to Optimizing Ms. Pac-Man Game Strategies in Real Time,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. PP, no. 99, pp. 1–1, 2016.
- [9] G. Recio, E. Martín, C. Estébanez and Y. Saez, “AntBot: Ant Colonies for Video Games,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, pp. 295–308, Dec 2012
- [10] Ms. Pac-Man vs. Ghost Competition, Simulator version [Online, last accessed 7.4.2016]. Available: <https://wcci12.herokuapp.com/>
- [11] Controllers from IEEE World Congress on Computational Intelligence (WCCI 2012) [Online, last accessed 5.5.2015]. Available: <https://wcci12.herokuapp.com/competitions/3>
- [12] The International Arcade Museum. Ms. Pac-Man [Online, last accessed 7.4.2016]. Available: http://www.arcade-museum.com/game_detail.php?game_id=8782
- [13] Java Genetic Algorithms Package (JGAP) [Online, last accessed 7.4.2016]. Available: <http://jgap.sourceforge.net/>
- [14] Controllers from IEEE Congress on Evolutionary Computation (CEC 2011) [Online, last accessed 5.5.2015]. Available: <https://wcci12.herokuapp.com/competitions/1>
- [15] L. Verhaard, “Overview of eiisolver Pacman/Ghosts” [Online, last accessed 7.4.2016]. Available: <https://github.com/eiisolver/pacman2012/blob/master/doc/pacman.pdf>
- [16] E. Voronyuk, “Ms Pac-man vs Ghosts Competition WCCI 2012” [Online, last accessed 7.4.2016]. Available: <http://brunneng.blogspot.ch/2012/05/ms-pac-man-vs-ghosts-competition.html>
- [17] D. Tose, “Artificial Intelligence” [Online, last accessed 7.4.2016]. Available: <http://www.blackmoor.org.uk/AI.htm>